LARA — All Experimental Results

Every run, its score, and what it established. Compiled 2026-09-05, updated 2026-09-07. Scores are recomputed from the stored run outputs, not copied from notes.

Full write-ups, if a section below is worth reading as a standalone paper: - Model Quality vs. the LARA Harness and Architecture — the full model × architecture × difficulty grid behind §2 below. - Does Separating Planning from Execution Help LLM Agents Solve Multi-App Tasks? (PDF) — the two-agent-split ablation behind §4 below; technical report: PLANNING_SEPARATION_ABLATION_2026-09-06.md. - Specialist-dispatch ablation (PDF) — behind §3 below.

---

1. Headline: the leaderboard entry

agent	model	split	tasks	TGC	SGC
LARA	`claude-opus-4-7`	`test_normal`	168	**88.7**	**82.1**
LARA	`claude-opus-4-7`	`test_challenge`	417	**85.6**	**77.0**

Merged to the AppWorld leaderboard 2026-09-03. **1st place on `test_challenge` as of 2026-09-06**, 12.2 TGC points clear of second (kecaipan capybara, 73.4). One Executor attempt per task, Reviewer and Supervisor retry both disabled, `appworld` 0.1.3.post1. The CI re-scored both bundles independently and reproduced these figures exactly.

---

2. Against the official baseline (`test_normal`, 168 tasks)

AppWorld ships a reference agent: the minimal ReAct loop from its own prompt template — no planning stage, no per-app knowledge, no cross-app memory. Official `appworld evaluate` scores, aggregate only.

`test_normal` (168 tasks):

agent	model	TGC	SGC	d1	d2	d3
official baseline	`gpt-4.1-mini`	23.2	7.1	47.4	18.8	4.8
official baseline	`claude-opus-4-7`	54.8	46.4	87.7	52.1	27.0
LARA	`gpt-4.1-mini`	61.9	50.0	91.2	52.1	42.9
LARA	`claude-opus-4-7`	**88.7**	**82.1**	98.2	89.6	79.4

`test_challenge` (417 tasks):

agent	model	TGC	SGC	d1	d2	d3
official baseline	`gpt-4.1-mini`	9.8	3.6	36.1	8.0	1.5
official baseline	`claude-opus-4-7`	27.3	18.7	75.0	26.7	10.3
LARA	`gpt-4.1-mini`	37.6	20.1	72.2	36.0	26.2
LARA	`claude-opus-4-7`	**85.6**	**77.0**	91.7	84.7	84.1

With the model held fixed, the architecture is worth:

split	model	baseline → LARA	TGC ratio	+TGC pts	SGC ratio
`test_normal`	`gpt-4.1-mini`	23.2 → 61.9	2.7×	+38.7	7.0×
`test_normal`	`claude-opus-4-7`	54.8 → 88.7	1.6×	+33.9	1.8×
`test_challenge`	`gpt-4.1-mini`	9.8 → 37.6	3.8×	+27.8	5.6×
`test_challenge`	`claude-opus-4-7`	27.3 → 85.6	**3.1×**	**+58.3**	4.1×

> **Conclusion — the architecture matters most exactly where the benchmark is hardest.** With all four cells filled, the pattern is not "the scaffold fades as the model improves." On the easier split a strong model does recover much of what the scaffold provides (1.6× on `test_normal`), but on `test_challenge` the same model without the scaffold collapses to 27.3 while LARA holds 85.6 — **3.1×, and +58.3 TGC points, the largest absolute gap of any cell measured.**
>
> The baseline drops 54.8 → 27.3 between splits (a 50% loss) while LARA drops only 88.7 → 85.6 (3.5%). Long multi-app tasks are where an unaided model runs out of steps rediscovering APIs, and where planning plus per-app knowledge pays off most.
>
> Reading the table the other way, the model is worth 26.8 TGC points on `test_normal` (61.9 → 88.7) and 48.0 on `test_challenge` (37.6 → 85.6). **Neither the architecture nor the model alone reaches the submitted score.**
>
> Difficulty-3 on `test_challenge` is the starkest figure in the project: the `gpt-4.1-mini` baseline scores **1.5 TGC / 0.0 SGC** — three tasks out of ~200, never once completing a full scenario — and even the `claude-opus-4-7` baseline manages only 10.3, where LARA on the same model holds **84.1 / 75.4**.
>
> **Failure mechanism.** On `test_challenge` 267 of 376 baseline failures (71%) and on `test_normal` 91 of 129 (70%) are `no_submit` — the agent exhausts its 16-step budget without calling `complete_task()`, spending it on discovering which APIs exist and hallucinating ones that do not (`set_access_token`, `show_account_usernames`, `list_transactions`). Removing that failure mode is what the Explorer stage is for.

---

3. Specialist-knowledge ablation (45-task train slice, `gpt-4.1-mini`)

Four arms, same slice, same day (2026-09-04), paired design. Only the Executor's system prompt varies between the middle three.

arm	specialist knowledge	prompt/ step	TGC	SGC	tok/task	solved/1M tok
baseline	none (no plan/ledger either)	—	13.3	4.0	64,752	2.1
generic	none	7,025 ch	64.4	56.0	**61,382**	**10.5**
dispatch (shipped)	current app only	~13,980 ch	75.6	68.0	77,818	9.7
monolith	all 10 apps	76,578 ch	**82.2**	**80.0**	189,666	4.3

Paired McNemar (exact, two-sided):

comparison	discordant	p	verdict
baseline vs dispatch	28 / 0	**< 0.0001**	significant — strict superset
generic vs monolith	9 / 1	**0.0215**	significant
generic vs dispatch	8 / 3	0.2266	not significant
dispatch vs monolith	3 / 6	0.5078	not significant

> **Conclusion 1 — the knowledge matters, the routing does not.** Accuracy is monotonic in how much specialist knowledge the Executor sees (64.4 → 75.6 → 82.2), and the largest knowledge gap is statistically significant. But *how* that knowledge is delivered — routed per step vs. concatenated flat — is indistinguishable (p = 0.51). The original hypothesis, that per-step routing improves accuracy by reducing distraction, is **not supported**.
>
> **Conclusion 2 — what routing actually buys is cost.** Dispatch matches monolith's accuracy at **2.44× fewer tokens** (3.39× on Executor input alone). Generic is the most token-efficient arm overall at 10.5 solved per million, reaching 85% of dispatch's accuracy for 79% of its cost.
>
> **Caveat.** Two identical monolith runs on consecutive days scored 75.6 and 82.2 — 6.6 points of pure run-to-run variance, **larger than the gap between arms**. No accuracy claim among generic/dispatch/monolith survives that; n=45 is underpowered for effects this size. The baseline gap (62 points, 28 discordant pairs) is far outside the noise band and is unaffected.

---

4. Planning-separation ablation (45-task train slice, same-day rerun 2026-09-06)

Does splitting Explorer from Executor beat one agent doing both? Superseded twice: an August v1 arm withheld the Explorer's knowledge and overstated the gap; v2 (2026-08-19) transplanted the knowledge live from `build_explorer_system()` but compared two runs from different sessions (64.4 vs 31.1). Given the drift finding in §5, both arms were **rerun same-day** on 2026-09-06 — see full paper linked above.

arm	TGC	SGC	tokens/task	notes
full LARA	**73.3** (33/45)	64.0	72,490	Explorer + Executor, 16 ReAct steps
single agent (v2)	42.2 (19/45)	40.0	76,560	knowledge-transplanted, 30 steps

Paired McNemar (exact): 18 tasks solved only by full LARA, 4 only by the single agent, 15 by both, 8 by neither — **p = 0.0043, significant**. 22 discordant pairs comfortably clears the "under 10 = directional only" bar that limited §3's arms — this is the most statistically robust result in this project.

> **Conclusion.** Separating planning from execution costs **31.1 TGC points** (73.3 → 42.2) when both arms are run same-day and knowledge is held fixed, and the effect is significant, not an artefact of comparing across sessions. Token cost is nearly identical between arms (72,490 vs 76,560, 5.6% apart) — full LARA is simultaneously cheaper *and* far more accurate. Difficulty-1 shows the largest absolute gap (92.3 vs 46.2), which argues the mechanism is not primarily about running out of steps on long tasks (confirmed directly: no failure in either arm occurred at that arm's step ceiling — see paper §5.1) but about something more basic in discovery-then-action versus discovery-interleaved-with-action.
>
> **The step-ordering confound was addressed, then tested, and found not to be driving the result.** `ablation_single_agent.py` now sorts the stub plan's app order by first-mention position in the task text rather than an arbitrary keyword-table order — a fix verified to

> actually change routing on 7 of 19 multi-app tasks in this slice. Isolating those 7 tasks specifically (pre-fix vs. post-fix, same code otherwise): **0 improved, 0 regressed** — the fix changed nothing on the tasks it targets. The 38 unaffected tasks moved by more (9 up, 2 down) from ordinary run-to-run noise alone. The 31.1-point gap above is not an ordering artefact; whatever the single agent is missing, it isn't (mainly) this.

---

5. Methodological finding: hosted-model drift

run	date	code	TGC
dispatch	2026-08-19	unchanged	64.4
dispatch	2026-09-04	unchanged	75.6

> **Conclusion.** The same code on the same 45 tasks scored **11.2 points higher two weeks later** on hosted `gpt-4.1-mini`. An earlier draft of §3 compared an August baseline against a September arm and concluded monolith *beat* dispatch by 11 points; re-running dispatch same-day erased the entire effect.
>
> **Any cross-day comparison against a hosted model is unsafe.** All §3 numbers are same-day. §4's original (August) run was not, which is exactly why it was rerun same-day on 2026-09-06 — the current §4 numbers are the corrected version and the August figures should not be quoted.

---

6. Token economics (measured, not estimated)

Provider `usage` figures recorded per call. Before this instrumentation no run had token accounting; estimates from prompt sizes undercounted by ~2.5×.

arm	tok/task	Executor input	Explorer input
baseline (`test_normal`)	62,361	—	—
baseline (`test_challenge`)	88,915	—	—
generic	61,382	~1.1M	~1.2M
dispatch	77,818	2.16M	1.27M
monolith	189,666	7.33M	1.13M

> **Conclusion.** Input dominates output ~48:1 — cost is what you put *into* context, not what the model writes. The **Explorer alone is 36% of dispatch's total input** from pre-injecting full API docs, making it the largest single optimisation target in the system and entirely unrelated to specialists. Injecting only the APIs the plan names is the obvious next experiment.

---

7. Compliance

- `login_to_app` and `find_contact` — helpers that called concrete endpoints from our own code — were removed 2026-08-10; authentication moved into the prompt, which the rules explicitly permit. Canned Amazon plans and an ACTION-task regex were deleted (coverage measurement showed they matched **zero** train/dev tasks).
- AST-verified: **zero `apis.*` calls in live code**; re-verified 2026-09-02.

- Held-out splits were scored, never analysed: no `tasks/<id>/evaluation/report.md` was opened for any `test_normal` run, and all ablation arms ran on train.

Appendix — where the data lives

run	directory
baseline, `test_normal`	`experiments/outputs/baseline_react_test_normal`
baseline, `test_challenge`	`experiments/outputs/baseline_react_test_challenge`
baseline, 45-task slice	`experiments/outputs/baseline_react_ext`
generic / dispatch / monolith	`experiments/outputs/spec_{generic,dispatch,monolith}_ext`
monolith, first run (no tokens)	`experiments/outputs/spec_monolith_ext_NOTOKENS`
planning ablation, full LARA (§4 headline)	`experiments/outputs/ablation_full_ext`
planning ablation, single-agent (§4 headline, 42.2%)	`experiments/outputs/ablation_single_ext_PRE_ORDERFIX`
planning ablation, single-agent (August, superseded)	`experiments/outputs/ablation_{full,single}_ext_AUG2026`
token logs	`tokens_*.jsonl`, `tokens_planning{full,single}_ext_PRE_ORDERFIX.jsonl`

⚠ **`experiments/outputs/ablation_single_ext` is NOT the §4 headline run.** After §4's numbers were generated, that directory was overwritten by a *diagnostic* rerun that tested the app-ordering fix in isolation (57.8% TGC — see the ordering-fix note in §4). The run the 42.2% figure and its p-value are computed from is preserved at `ablation_single_ext_PRE_ORDERFIX`; use that path, not the bare `_ext` one, to reproduce §4.

Note on difficulty columns. Per-difficulty figures in §2 come from the official `appworld evaluate` report. The local `analysis/specialist_ablation_report.py` infers difficulty from the task-id suffix, which is the *variant* number — its per-difficulty output is not comparable to the official breakdown and should not be quoted.